ECAT Master STM32H7 开发板
使用说明 V1.1

保密

目录

1.技术支持联系方式.....3
2.文档说明.....3
3.硬件.....3
4.软件程序说明.....6
5.软件使用说明.....8
5.1 IO 类从站.....8
5.1.1 普通 IO 从站.....8
5.2 电机类从站.....10
5.2.1 相关函数参数设置.....11
5.2.2 从站 MAX/ID/NAME 设置（必须对应）.....11
5.2.3 对象字典映射.....15
5.2.4 定义输入输出结构体.....20
5.2.5 发送控制指令.....20
5.3 混合（电机+IO）从站.....24
5.4 EtherCAT+TCP/IP 例程.....25
6.其他.....27

保密

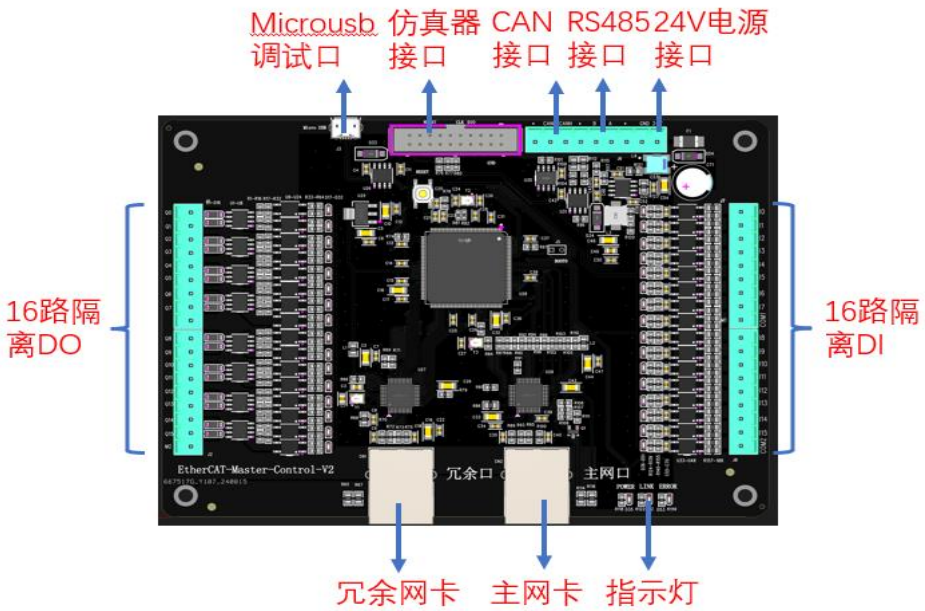
1.技术支持联系方式

首先感谢您购买合妍科技 EtherCAT　Master STM32H7 开发板，假如开发板有问题可提供免费维修。请您务必先阅读该文档，该文档是我司技术人员多年的经验总结和提炼，绝大部分参考该文档就能解决。有任何技术问题可联系旺旺获取技术支持方式，将为您提供全面的技术支持。

2.文档说明

本文档针对开发板的例程做一个详细讲解，以方便更快的熟悉整个软件流程。此文档能够支撑调通 ethercat 伺服、步进或 IO 从站。若有任何问题请添加技术支持微信或 QQ，将为您提供技术支持服务。

3.硬件



如图所示，此开发板可通过 USB 线插到电脑上供电，也可以直接使用 JTAG 调试口供电。开发板要正常与从站通讯运行，必须在主网口上插上网线，若只在主网口插网线，则开发板工作在单网模式下。若主网口和冗余网口都插网线，则开发板工作在冗余模式下。若在上电前只使用冗余网口连从站是无法工作的，在

冗余模式下，拔掉主网口，只留冗余口是能够工作的。开发板上电前需先把从站设备上电，把网线插好。

主站开发板的工作模式：

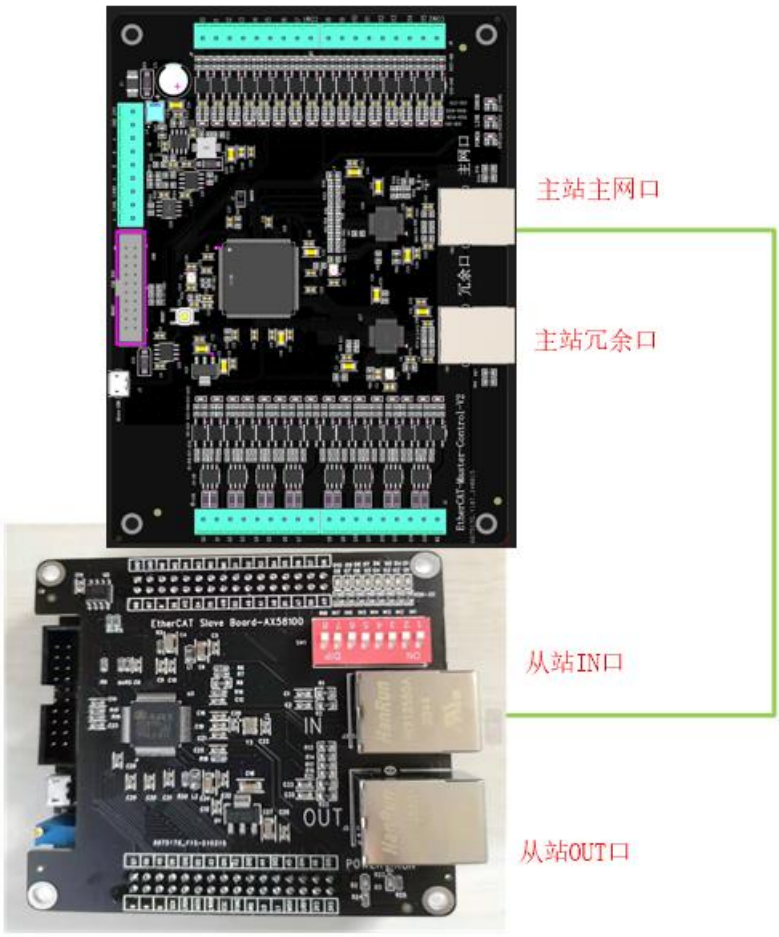


图 1　单网模式 可以正常工作

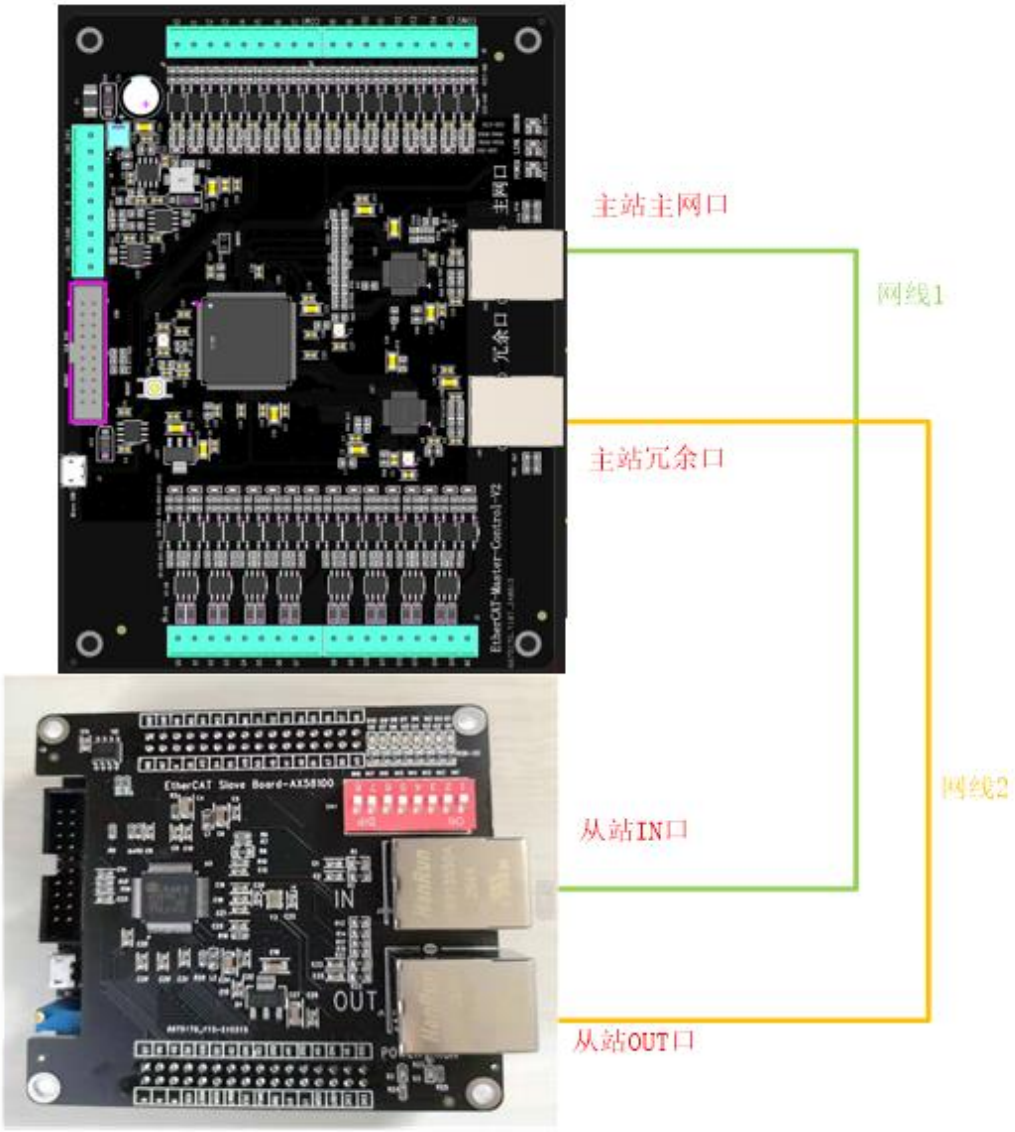


图 2　冗余模式

主站开发板上电工作于冗余后，不管是网线 1 或者网线 2 单独断掉，主站仍然可以正常工作。该模式下即体现了冗余的优势。

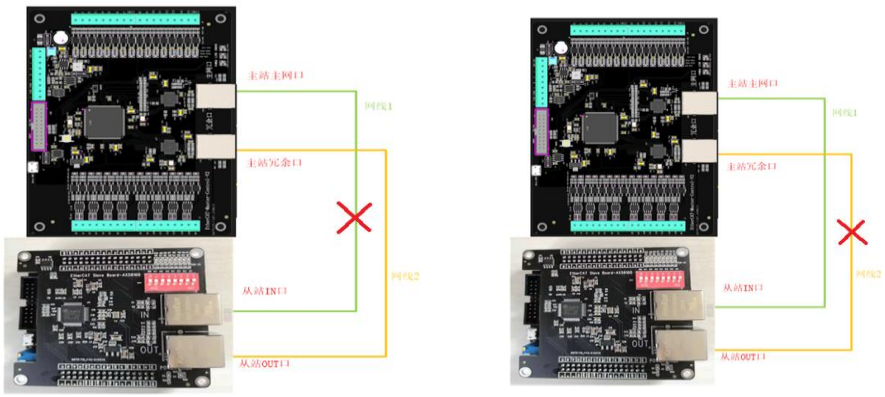


图 3 非正常模式

使用冗余口在上电前单独连接从站，主站开发板是无法正常工作的。

开发板配套的 USB 线支持串口通讯，调试时可打印相应的调试信息，辅助调试。串口参数 115200，N,8,1。

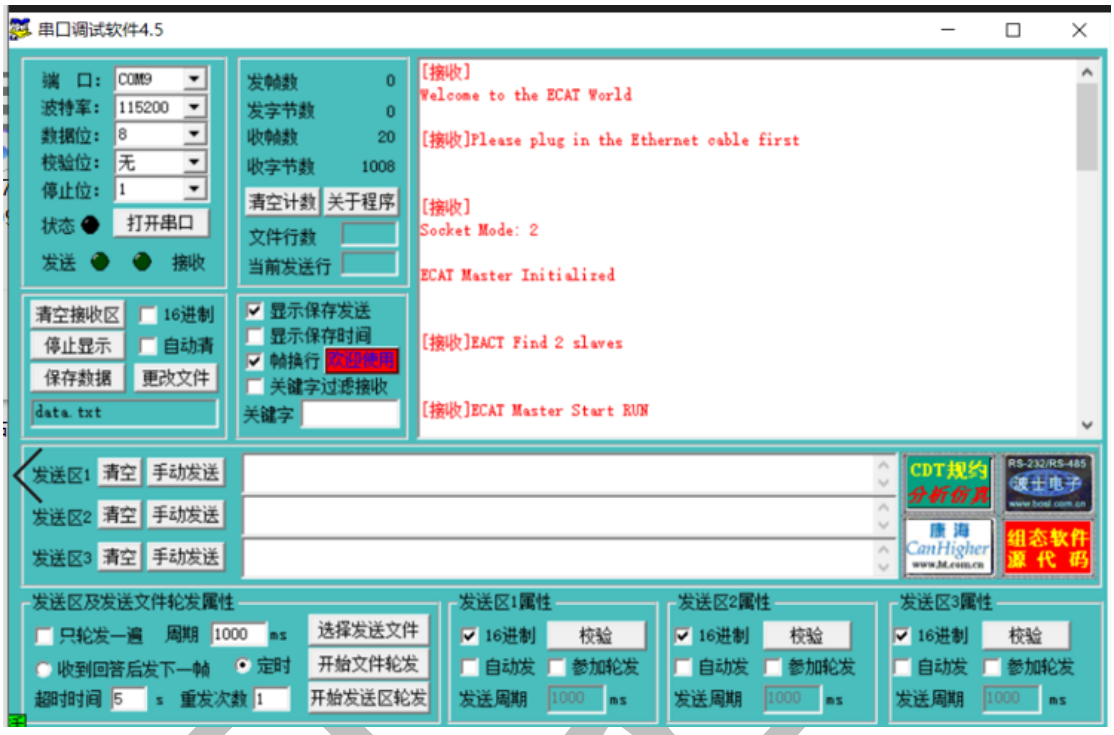
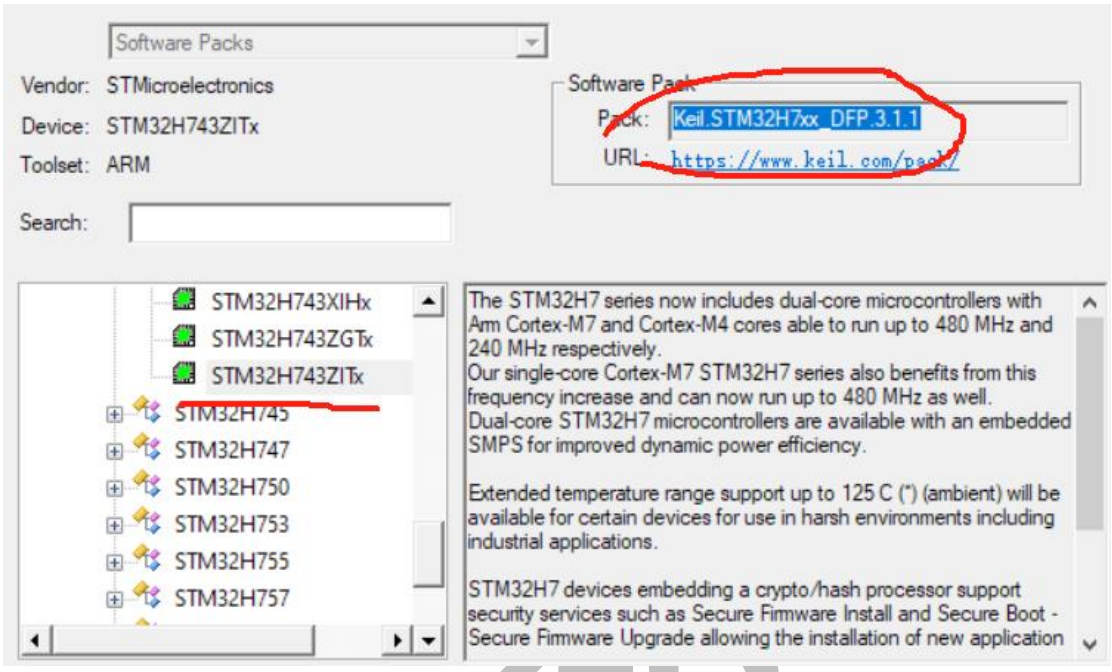


图 3　调试口打印信息示例

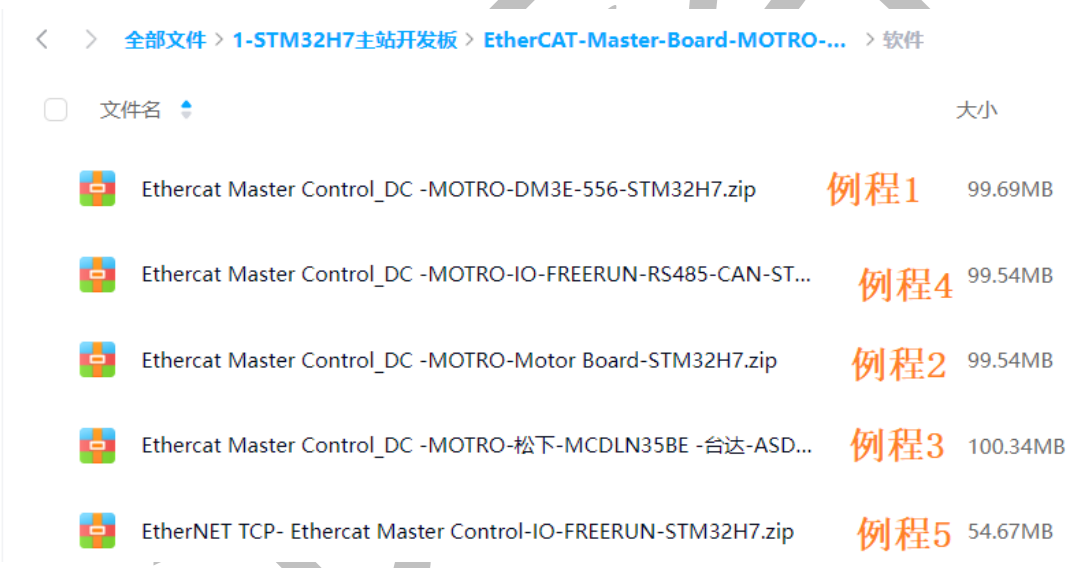
4.软件程序说明

Keil 版本开发平台需求 Keil V5.24,DFP 版本为 DFP.2.15.0，一定保证，不然低版本可能会出现以太网驱动等不兼容，匹配的现象。





软件例程说明：（重要，请仔细阅读）



如上图，主站开发板中提供了众多的源码例程，例程所在的位置目录：EtherCAT-Master-Board-MOTRO-STM32H7/软件。

下面对例程的情况逐一进行说明：

例程 4：出厂默认烧写程序。该例程主要用于控制从站 IO（即从站是一些简单的 IO 组成的），也可以直接用于控制我们的从站开发板。市面上大部分从站 IO 可以直接控制。从站 IO（即一个从站对应于固定的 IO 点数，如 16DI/16DO 等），可直接对 IO 从站进行扫描，进入 OP。若有多个从站，需注意区分每个从站对应的 IO 起始字节。

例程 2：用于控制伺服的例程。可直接用于控制我们的伺服开发板。

例程 1：用于控制伺服的例程。该例程为本文档中用于举例的例程，用于说明对不同的伺服应该怎么修改 PDO 配置和参数。

例程 3：用于控制多个伺服的例程。该例程主要控制了两个伺服。若有控制 2 个以上伺服的需求，参考该例程即可完成对多个伺服的控制。

例程 5：只有单个网口工作于 ethercat 模式，另一个网口工作于 tcpip 模式。例程是主网口工作于 tcpip 模式，传输 tcp 或 udp 数据，而冗余口工作于 ethercat 模式。可以通过连接以太网的上位机把数据传到主站开发板，主站开发板再控制伺服或者从站 IO 设备。

若从站为伺服或步进驱动器请优先参考例程 1，若从站为 IO 请优先参考例程 4。若既有伺服步进又有 IO 类从站，那么用例程 1 把伺服添加后，IO 添加对象字典与伺服一样添加就行。

5.软件使用说明

5.1 IO 类从站

5.1.1 普通 IO 从站

普通 IO 从站直接运行主站程序，对扫描到的每一个从站的字节注意区分起始位置就行。程序中只做了一个从站的 IO 输入输出。需要操作多个从站可仿照例程进行修改。

1. xml 文件介绍

XML 文件作用是描述从站设备，通过对 xml 文件的详细了解，才能做到无需上位机软件配置，直接写对应的控制指令。如需有上位机进行配置，也可以通过了解 xml 文件的结构后，从上位机发配置信息，与 stm32 通讯，对从站做出配置。

xml 文件可通过：EtherCAT-Master-Board-MOTRO-STM32H7\工具目录下的 XMLnotepad 软件打开、编辑。

2. 软件关键程序介绍

保密

DM9000_Init(); //DM9000 网口初始化

DM9162_Init(); //DM9162 网口初始化

上面两个函数对两个网口进行初始化，网络没有使用任何协议栈，直接操作网络相关的寄存器，这样能够保证以最快的速度收发以太网数据包。

```
//双网冗余运行模式
    if((ETH_ReadPHYRegister(DM9162_PHY_ADDRESS, PHY_BSR) & 0x0004)&&(dm9k_ReadReg(DM9000_REG_NSR)
      {
       socket_mode=2;
      }
//单网运行模式
    else if((ETH_ReadPHYRegister(DM9162_PHY_ADDRESS, PHY_BSR) & 0x0004)&&(!(dm9k_ReadReg(DM9000_RE
     {
       socket_mode=1;
     }
//其他情况
    else
     {
      socket_mode=0;
     }
    do
    {
      printf("Please plug in the Ethernet cable first \n\r");
      Delay_ms(100);
      bsp_LedToggle(2);    // 错误灯闪
    }while(socket_mode==0); //检测到错误不允许运行
```

对网口是否连网线进行了检测，判断运行于什么状态。若无网络或主网口没协商通过，那么故障灯一直闪烁。不允许运行后面的程序。

ec_init();//配置单网运行 or 双网冗余运行

该函数对网口进行配置

```
int ec_init(void)  //更改：不需要传参数
{
   if(socket_mode==1)
     return ecx_init(&ecx_context);
   else if(socket_mode==2)
     return ecx_init_redundant(&ecx_context,&ecx_redport);
}
```

两种模式配置的方式是不同的函数，请仔细看相关函数的区别。

ec_config_init（）

初始化，搜索从站模块，如果小于 0 则返回，大于 0 则用下面的函数对从站模块进行配置。

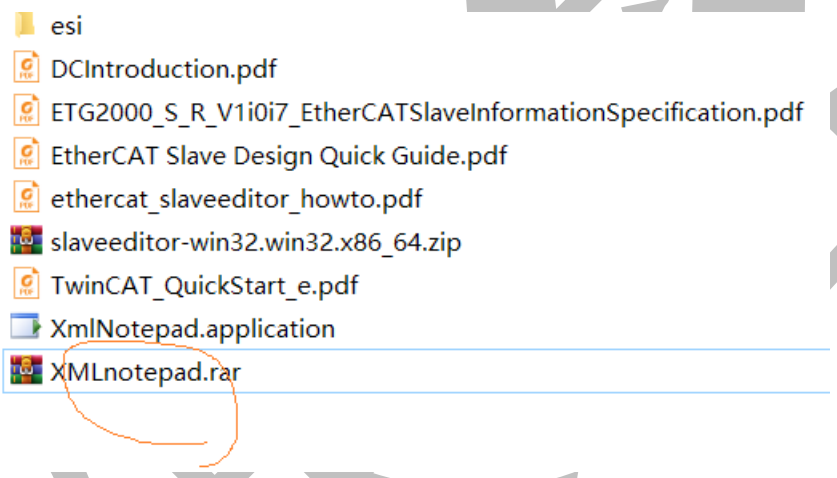
ec_config_map()

对从站模块进行配置

对从站模块的配置，主要是配置 FMMU，和 SM 相关参数。FMMU 的起始地址，长度，起始字节，结束字节，物理地址等等。

5.2 电机类从站

在进行说明前，请先安装打开 xml 文件的软件。Xml 文件有很多软件都可以打开，为便于对应，请安装工具目录下的该软件。



解压后选择 msi 尾缀的文件进行安装。准备工具就绪后，打开伺服对应的 xml 文件。



对使用主站控制电机做一个简单的程序说明，以便能够快速的上手。

电机类从站与 IO 类从站不太一样，一般 IO 类从站的 IO 数据是固定的，对象字典也是固定的，不需要重新设置对象字典，直接扫描即可。

注意：上电顺序，请先给伺服上电，后给开发板上电。在调试过程中，伺服

可能会报错，如果伺服有报错的情况，务必重启伺服，因为有些故障无法自动消除。

电机类从站（遵循 CIA402 标准的伺服或者步进控制系统以下统称为电机类从站）需要修改的地方：

5.2.1 相关函数参数设置

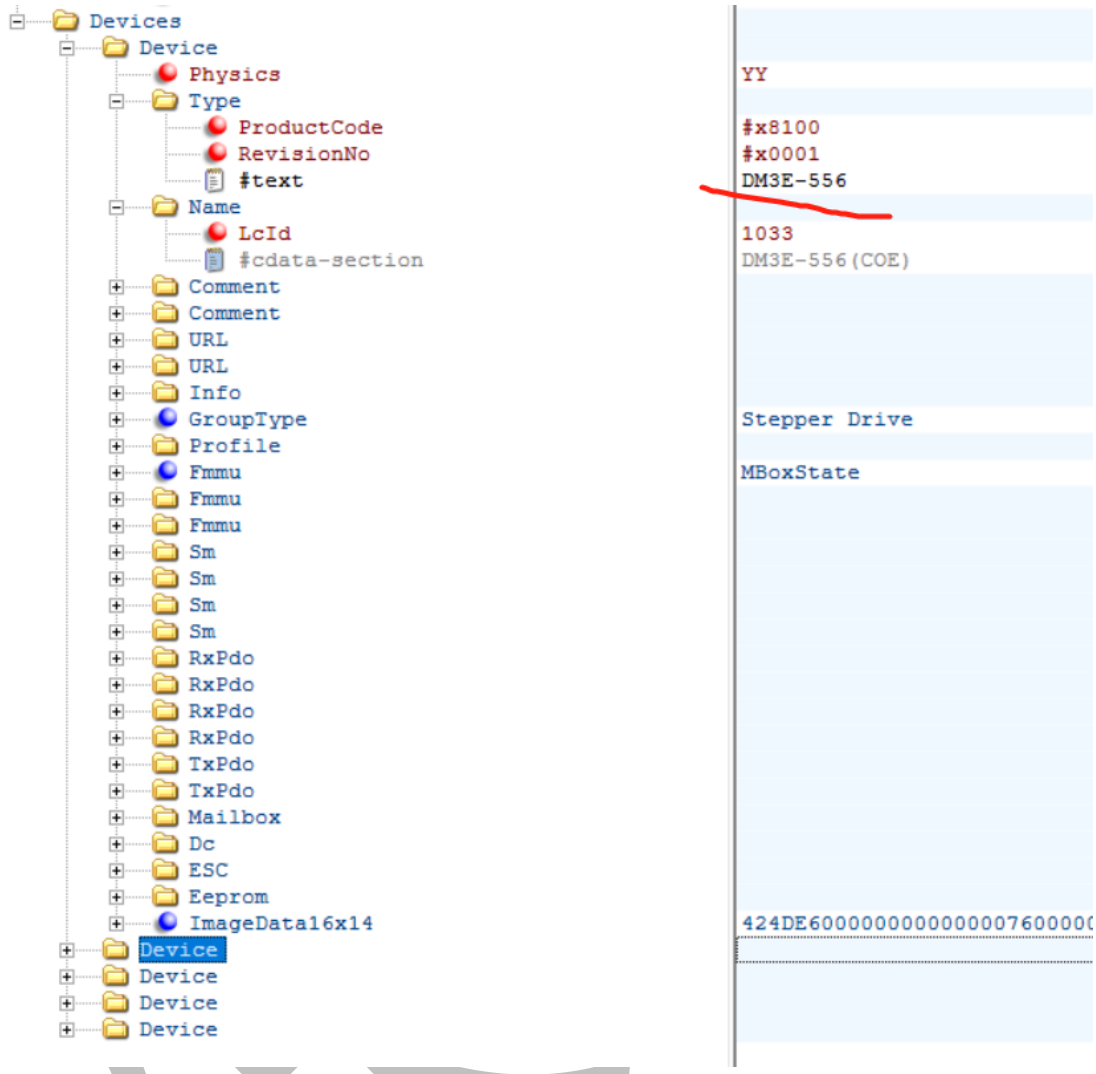
如下图，参数传递为 TRUE，此处无需修改。

```
    if ( ec_config_init(TRUE) > 0 )
//    if ( ec_config_init(FALSE) > 0 )
```

5.2.2 从站 MAX/ID/NAME 设置**（必须对应）**

Xml 文件有多个 DEVICE 时，请先找到实际对应硬件的型号，XML 如下图，有 6 个 DEVICE,我们找到其中与硬件对应的型号，DM3E-556。

保密

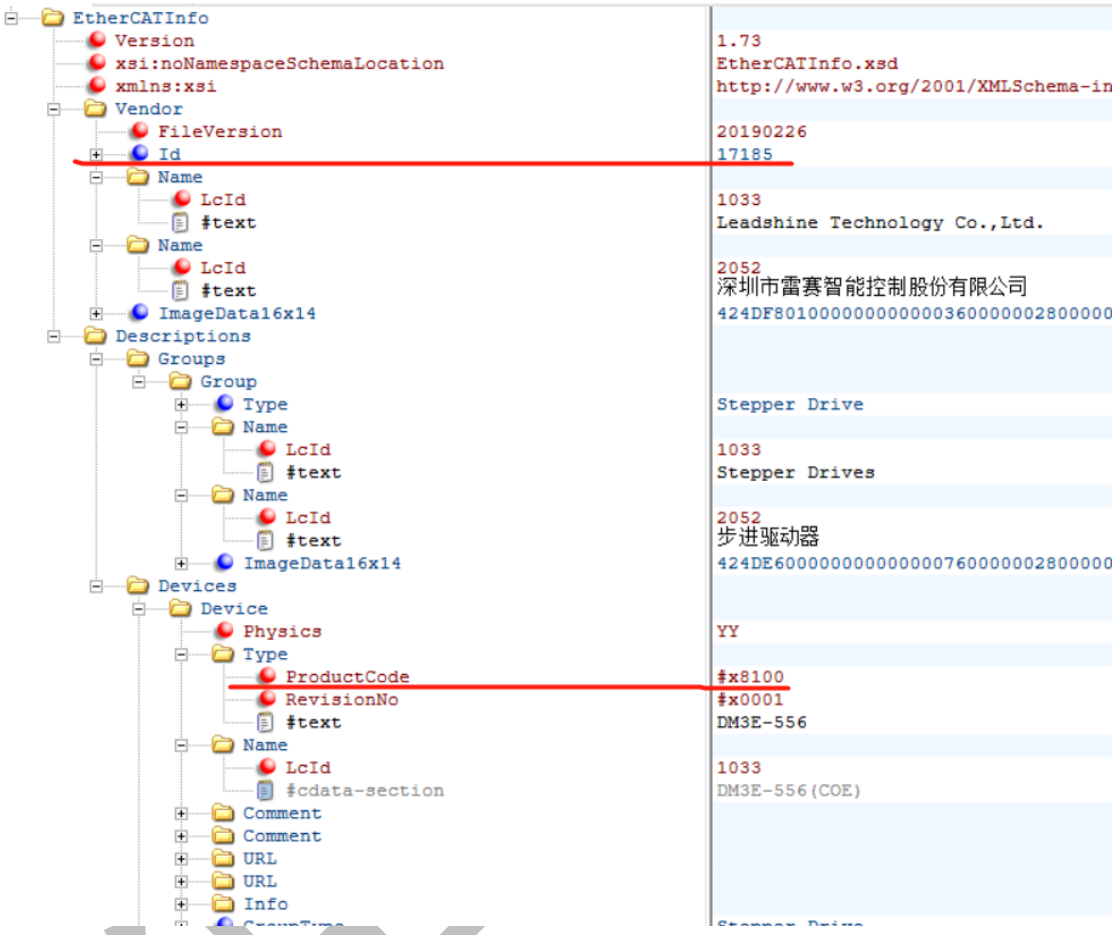


我们需要修改 ethercatconfiglist.h 文件中的如下地方，对应上图中 XML 文件找到的 DEVICE 进行。如下图中第一行和第三行不能改，若需要配置从站从第二行开始，有多个从站就配置如第二行的数据，将第二行的数据复制粘贴为第三行，配置多个。下图为配置了雷赛 DM3E-556。

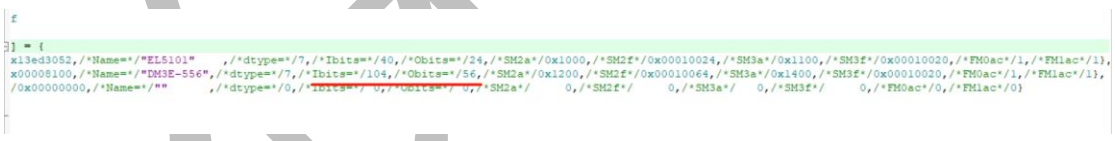


，ProductCode 对应上述函数中的 ID，

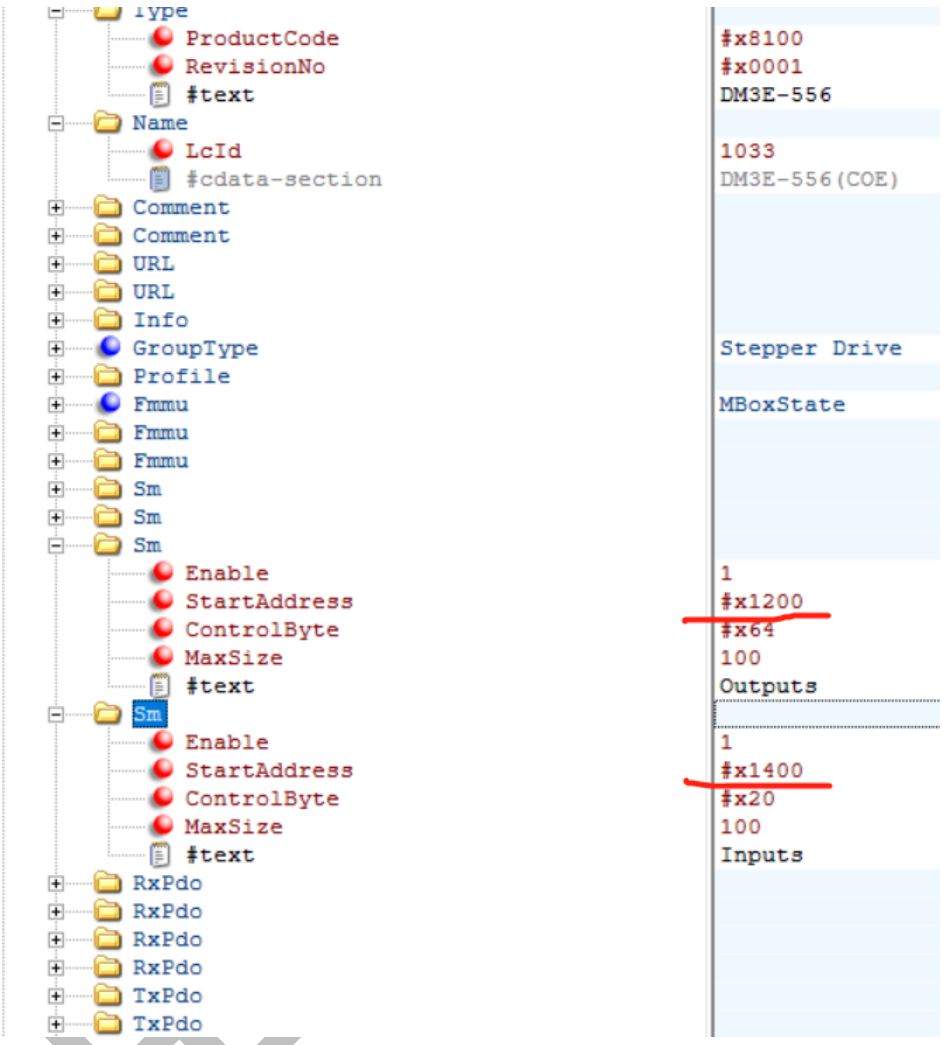
xml 中的 ID 对应填到 MAN 的地方



Ibit 和 Obit 比较关键，后面再做介绍。**（第一步调试的时候不用关心这个地方）**



sm2a,sm3a 分别对应 xml 中的第三个和第四个 SM,该处一定与 XML 文件保持一致，不然会报错。



下面这几个参数无需进行修改。其余的 sm2f 是 enable 和 controlbyte 的组合，即 enable 在前 controlbyte 在后 0x00010064，sm3f 类似，参照上图填写。

FM0AC 和 FM1AC 填 1 即可。

Dtype 填 7 即可。

注意注意：

因为我们程序使用了在 PDO 配置的地方使用配置动态 PDO（0x1600 和 0x1a00）的方法，所以绝大部分电机类从站只修改上述几处地方后就可以进入 OP 状态了。到此处后就可以编译下载到开发板中运行，然后看串口打印的调试信息是否进入 OP 了。若未进入 OP，请先检查核对上面几个参数是否正确，若确认

无误，仍然报故障，那么可能该从站不支持这种方法（具体是否支持，请参考伺服手册），请继续按照后面的章节进行操作。

5.2.3 对象字典映射

不同的控制对象需要对应不同的对象字典，此处需要查看手册和 xml 文件做对应的配置。

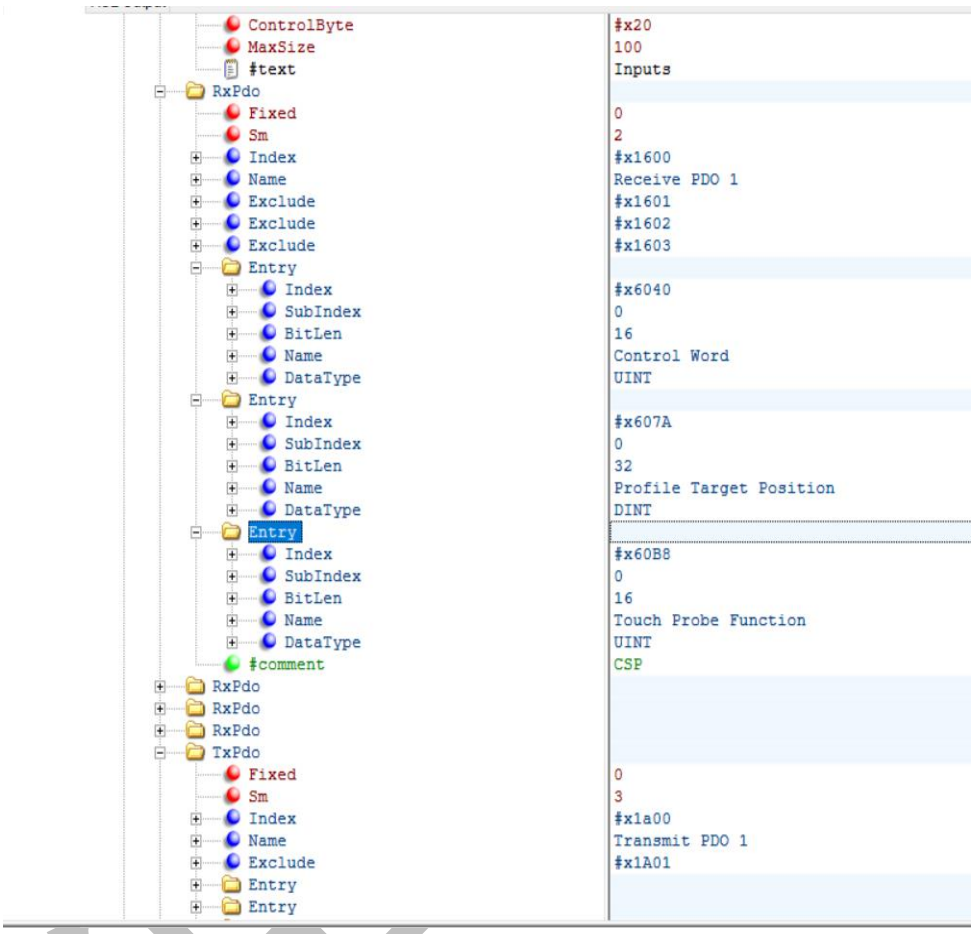
程序所在位置如下图：

```
160       MAC_Init(); //DM9162参数初始化
161       ec_init();//配置单网运行or双网冗余运行
162
163       printf("\r\nSocket Mode: %d\n\r", socket_mode);
164       bsp_LedOn(1);   //  Run on
165     if(ec_config_init(TRUE)>0)
166       {
167         printf("%d slaves found and configured.\r\n",ec_slavecount);
168         if((ec_slavecount >= 1))
169           {
170           for(slc = 1; slc <= ec_slavecount; slc++)
171             {
172                   // beckhoff EL7031, using ec_slave[].name is not very reliable
173 //                if((ec_slave[slc].eep_man == 0x00850104) && (ec_slave[slc].eep_id == 0x01030507)
174 //                {
175                   printf("Found %s at position %d\n", ec_slave[slc].name, slc);
176 //                  u8val = 8;
177 //                  ec_SDOwrite(slc, 0x6060, 0x00, FALSE, sizeof(u8val), &u8val, EC_TIMEOUTRXM);
178 //                }
179               ec_slave[slc].PO2SOconfig = &Servosetup;
180               }
181             }
182           /* Run IO mapping */
183
184          ec_configdc();
185          ec_dcsync0(1, TRUE, SYNC0TIME, 250000); // SYNC0 on slave 1
186          ec_config_map(&IOmap);
```

```
51  uint8  sdakaka[128];
52  int Servosetup(uint16 slave)
53  {
54      int retval;
55      uint16 u16val;
56      uint8  u8val;
57      uint32 u32val;
58      retval = 0;
59
60      u8val = 0;
61      retval += ec_SDOwrite(slave, 0x1c12, 0x00, FALSE, sizeof(u8val), &u8val, EC_TIMEOUTRXM);
62      u16val = 0x1600;
63      retval += ec_SDOwrite(slave, 0x1c12, 0x01, FALSE, sizeof(u16val), &u16val, EC_TIMEOUTRXM);
64      u8val = 1;
65      retval += ec_SDOwrite(slave, 0x1c12, 0x00, FALSE, sizeof(u8val), &u8val, EC_TIMEOUTRXM);
66
67
68      u8val = 0;
69      retval += ec_SDOwrite(slave, 0x1600, 0x00, FALSE, sizeof(u8val), &u8val, EC_TIMEOUTRXM);
70      u32val = 0x60400010;
71      retval += ec_SDOwrite(slave, 0x1600, 0x01, FALSE, sizeof(u32val), &u32val, EC_TIMEOUTRXM);
72      u32val = 0x607A0020;
73      retval += ec_SDOwrite(slave, 0x1600, 0x02, FALSE, sizeof(u32val), &u32val, EC_TIMEOUTRXM);
74      u32val = 0x60600008;
75      retval += ec_SDOwrite(slave, 0x1600, 0x03, FALSE, sizeof(u32val), &u32val, EC_TIMEOUTRXM);
76      u8val = 3;
77      retval += ec_SDOwrite(slave, 0x1600, 0x00, FALSE, sizeof(u8val), &u8val, EC_TIMEOUTRXM);
78
79
80      u8val = 0;
```

上面的配置参数均为动态配置的，之所以能采用动态配置是因为伺服支持。

若有的伺服不支持动态配置，则完全按照 xml 文件上的配置进行即可。



我们可以看到 xml 文件中有很多个 RXpdo 和 TXpdo，一般情况下，我们需要哪种对象字典就选用哪个即可。TXPDO 和 RXPDO 我们各配置一种。

对 PDO 进行配置，分别配置输入和输出。上面填写的 Ibit 和 Obit 与此处重映射的输入输出有关。一定得对应上。不然主站无法进入 OP 状态。

PDO 映射用于建立对象字典 PDO 的映射关系。1600h-17FFh 为 RPDO,1A00h-1BFFh 为 TPDO，不同的厂家会定义不同的映射。需要注意是可变映射还是固定映射，可变映射对象字典的内容可以变化。固定映射则不能。

PDO 映射参数包含指向 PDO 需要发送或者接收到的 PDO 对应的过程数据的指针，包括索引、子索引及映射对象长度。其中子索引 0 记录该 PDO 具体的对象个数 N，每个 PDO 数据长度最多可达 4*N 个字节，可同时映射一个或者多

个对象。子索引 1-N 则是映射内容。映射参数内容定义如下。

位数	31	……	16	15	……	8	7	……	0
含义	索引			子索引			对象长度		

索引和子索引共同决定对象在对象字典中的位置，对象长度指明该对象的具体位长，用十六进制表示，即：

对象长度	位长
08h	8 位
10h	16 位
20h	32 位

例如表示 16 位控制字 6040h-00 的映射参数为 60400010h。

简要介绍一下 u32val 这个变量后面跟的数的含义，以 0x60400010 为例，其中 0x6040 为该对象字典，接下来 00 为该对象字典的 subitem，最后 10 为该对象字典的 BIT 长度，0x10 即为 16BIT,2 个 byte。

如上 0x1603 的计算 OBIT=0x10+0x20+0x10+0x20+0x20+0x20+0x10=176。

IBIT 的计算类似。

下面详细讲一下对象字典的配置，参考的文件《DM3E 系列设备描述文件_V1.75.xml》:

先配置 0x1c12 和 0x1c13，0x1c12 对应 0x16xx，0x1c13 对应 0x1axx。

```
int retval;
uint16 u16val;
uint8  u8val;
uint32 u32val;
retval = 0;

u8val = 0;
retval += ec_SDOwrite(slave, 0x1c12, 0x00, FALSE, sizeof(u8val), &u8val, EC_TIMEOUTRXM);
u16val = 0x1600;
retval += ec_SDOwrite(slave, 0x1c12, 0x01, FALSE, sizeof(u16val), &u16val, EC_TIMEOUTRXM);
u8val = 1;
retval += ec_SDOwrite(slave, 0x1c12, 0x00, FALSE, sizeof(u8val), &u8val, EC_TIMEOUTRXM);
```

索引	子索引	内容
0x1C12	01h	选择使用 0x1600 等中的一个作为实际使用的 RPDO
0x1C13	01h	选择使用 0x1A00 等中的一个作为实际使用的 TPDO

PDO 的映射配置遵循特定的流程，具体按如下步骤执行：

1.无效 PDO。1C12h（或 1C13h）的 00h 子索引写入“0”即可清除该 PDO 原有的所有映射；

写入 PDO 映射内容。按映射定义分别写入映射参数子索引 1-10；

写入该 PDO 映射对象总个数。

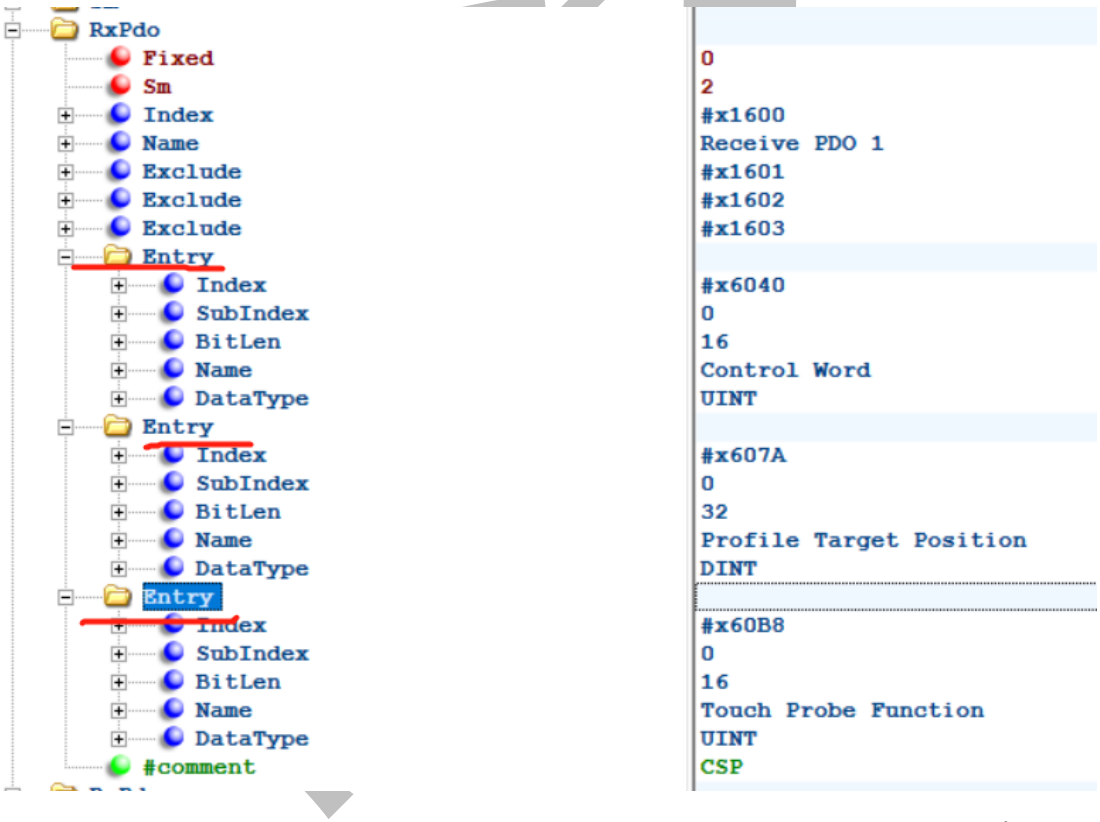
2.有效 PDO.1C12h（1C13h）的 00h 子索引写入 1。

操作步骤，先把 0x1c12 的子项 0 清 0，操作数据为 8 位，然后给 0x1c12 的第一个子项配置对象字典，如上配置的 0x1600，然后写 0x1c12 的子项 0 为 1，表示 0x1c12 配置了 1 个对象字典。0x1c13 的配置方式类似。此处可配置的项为 0x1600,0x1601,0x1602，0x1603 详见 xml 文件。这里根据需要选择配置上述 4 个对象字典。

保密



下面介绍配置 0x1600 的内容：把 xml 中 RxPdo 下面的 Entry 配置到从站里面去。



0x1600 对象字典中包含三个子项，那么我们就需要配置三个子项。如下图，操作步骤，首先把 0x1600 的子项 0 清 0，然后依次配置 3 个子项，最后把 0x1600 的子项 0 配置为 3。u32val 这个变量后面跟的数的含义，注意这是一个 32 位的数，以 0x60400010 为例，其中 0x6040 为该对象字典，接下来 00 为该对象字典的 subitem，最后 10 为该对象字典的 BIT 长度，0x10 即为 16BIT（即上图中的 BitLen），2 个 byte。0x1A00 的配置方式类似。

```
u8val = 0;
retval += ec_SDOwrite(slave, 0x1600, 0x00, FALSE, sizeof(u8val), &u8val, EC_TIMEOUTRXM);
u32val = 0x60400010;
retval += ec_SDOwrite(slave, 0x1600, 0x01, FALSE, sizeof(u32val), &u32val, EC_TIMEOUTRXM);
u32val = 0x607A0020;
retval += ec_SDOwrite(slave, 0x1600, 0x02, FALSE, sizeof(u32val), &u32val, EC_TIMEOUTRXM);
u32val = 0x60600008;
retval += ec_SDOwrite(slave, 0x1600, 0x03, FALSE, sizeof(u32val), &u32val, EC_TIMEOUTRXM);
u8val = 3;
retval += ec_SDOwrite(slave, 0x1600, 0x00, FALSE, sizeof(u8val), &u8val, EC_TIMEOUTRXM);
```

可以看到程序中配置的三个对象字典与 xml 文件中 0x1600 下面的内容并不一致。这是对对象字典进行了重映射，能否重映射，哪些对象字典能重映射得参考伺服的手册说明书（注意有些厂家手册并没有说明能不能重映射）。

5.2.4 定义输入输出结构体

此处需要与上面填写的对象字典的个数和数据类型对应上。

```
94  typedef __packed struct
95  {
96      uint16 ControlWord;
97      int32 TargetPos;
98      uint8 TargetMode;
99  }PDO_Output;
100
101 typedef __packed struct
102 {
103     uint16 StatusWord;
104     int32 CurrentPosition;
105     int32 CurrentVelocity;
106     uint16 ErrorCode;
107     uint8 CurrentMode;
108 }PDO_Input;
109 void ecat_loop(void);
110
```

5.2.5 发送控制指令

发送控制字使驱动器进行状态切换。驱动器状态切换如下图。

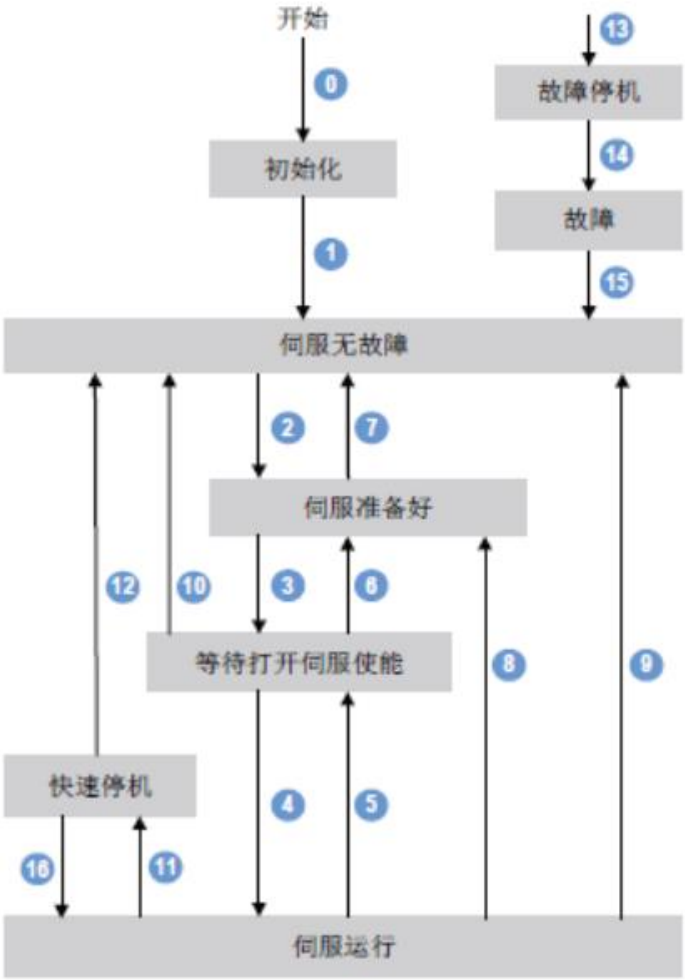


图 7-21 CiA402 状态机切换图

初始化	驱动器初始化、内部自检已经完成。驱动器的参数不能设置，也不能执行驱动功能。
伺服无故障	伺服驱动器无故障或错误已排除。驱动器参数可以设置
伺服准备好	伺服驱动器已准备好。驱动器参数可以设置。
等待打开伺服使能	伺服驱动器等待打开伺服使能。驱动器参数可以设置。
伺服运行	驱动器正常运行，已使能某一伺服运行模式，电机已通电，指令不为0 时，电机旋转。驱动器参数属性为“运行更改”的可以设置，其他不可。
快速停机	快速停机功能被激活，驱动器正在执行快速停机功能。驱动器参数属性为“运行更改”的可以设置，其他不可。
故障停机	驱动器发生故障，正在执行故障停机过程中。驱动器参数属性为“运行更改”的可以设置，其他不可。
故障	故障停机完成，所有驱动功能均被禁止，同时允许更改驱动器参数以便排除故障。

CiA402 状态切换		控制字 6040h	状态字 6041h 的 bit0~bit9*1
0	上电 → 初始化	自然过渡，无需控制指令	0x0000
1	初始化 → 伺服无故障	自然过渡，无需控制指令 若初始化中发生错误，直接进入 13	0x0250
2	伺服无故障 → 伺服准备好	0x0006	0x0231
3	伺服准备好 → 等待打开伺服使能	0x0007	0x0233
4	等待打开伺服使能 → 伺服运行	0x000F	0x0237
5	伺服运行 → 等待打开伺服使能	0x0007	0x0233
6	等待打开伺服使能 → 伺服准备好	0x0006	0x0231
7	伺服准备好 → 伺服无故障	0x0000	0x0250
8	伺服运行 → 伺服准备好	0x0006	0x0231
9	伺服运行 → 伺服无故障	0x0000	0x0250
10	等待打开伺服使能 → 伺服无故障	0x0000	0x0250
11	伺服运行 → 快速停机	0x0002	0x0217
12	快速停机 → 伺服无故障	快速停机方式 605A 选择为 0~3，停机完成后，自然过渡，无需控制指令	0x0250
13	→ 故障停机	除“故障”外其他任意状态下，伺服驱动器一旦发生故障，自动切换到故障停机状态，无需控制指令	0x021F
14	故障停机 → 故障	故障停机完成后，自然过渡，无需控制指令	0x0218
15	故障 → 伺服无故障	0x80 bit7 上升沿有效； bit7 保持为 1，其他控制指令均无效。	0x0250
16	快速停机 → 伺服运行	快速停机方式 605A 选择为 5~7，停机完成后，发送 0x0F	0x0237

序号	描述	6040控制字	6041状态字
2	伺服无故障 伺服准备好	0x0006	0x0231
3	伺服准备好 等待打开伺服使能	0x0007	0x0233
4	等待打开伺服使能 伺服运行	0x000F	0x0237

也就是向6040PDO中依次写入6-7-15（要根据6041状态字反馈信号依次写入）

也就是向6040PDO中依次写入6-7-15（要根据6041状态字反馈信号依次写入）

6040控制字寄存器各个位描述

索引 6040h	名称	控制字 control word			设定生效	运行设定停机生效	数据结构	VAR	数据类型	Uint16
	可访问性	RW	能否映射	RPDO	相关模式	ALL	数据范围	0~65535	出厂设定	0

设置控制指令：

bit	名称		描述
0	可以开启伺服运行	switch on	1- 有效，0- 无效
1	接通主回路电	enable voltage	1- 有效，0- 无效
2	快速停机	quick stop	0- 有效，1- 无效
3	伺服运行	enable operation	1- 有效，0- 无效
4~6	运行模式相关	operation mode specific	与各伺服运行模式相关。
7	故障复位	fault reset	对于可复位故障和警告，执行故障复位功能 bit7 上升沿有效； bit7 保持为 1，其他控制指令均无效。
8	暂停	halt	各模式下的暂停方式请查询对象字典 605Dh。
9	运行模式相关	operation mode specific	与各伺服运行模式相关。
10	保留	reverse	未定义
11~15	厂家自定义	manufacturer-specific	厂家自定义

◆注意：

控制字的每一个 bit 位单独赋值无意义，必须与其他位共同构成某一控制指令；

bit0~bit3 和 bit7 在各伺服模式下意义相同，必须按顺序发送命令，才可将伺服驱动器按照 CiA402 状态机切换流程引导入预计的状态，每一命令对应一确定的状态；

bit4~bit6 与各伺服模式相关（请查看不同模式下的控制指令）；

bit9 未定义功能。

6041状态字寄存器各个位描述

7.2.2 状态字 6041h

索引 6041h	名称	状态字 status word			设定生效	-	数据结构	VAR	数据类型	Uint16
	可访问性	RO	能否映射	TPDO	相关模式	ALL	数据范围	0~xFFFF	出厂设定	0

反映伺服状态：

bit	名称		描述
0	伺服准备好	ready to switch on	1- 有效，0- 无效
1	可以开启伺服运行	switch on	1- 有效，0- 无效
2	伺服运行	operation enabled	1- 有效，0- 无效
3	故障	fault	1- 有效，0- 无效
4	主回路电接通	voltage enabled	1- 有效，0- 无效
5	快速停机	quick stop	0- 有效，1- 无效
6	伺服不可运行	switch on disabled	1- 有效，0- 无效
7	警告	warning	1- 有效，0- 无效
8	厂家自定义	manufacturer-specific	未定义功能
9	远程控制	remote	1- 有效，控制字生效 0- 无效
10	目标到达	target reach	1- 有效，0- 无效
11	内部限制有效	internal limit active	1- 有效，0- 无效
12~13	运行模式相关	operation mode specific	与各伺服运行模式相关。
14	厂家自定义	manufacturer-specific	未定义功能
15	原点已找到	Home Find	1- 有效，0- 无效

设定值（二进制）	描述
xxxx xxxx x0xx 0000	未准备好（Not ready to switch on）
xxxx xxxx x1xx 0000	启动失效 (Switch on disabled)
xxxx xxxx x01x 0001	准备好 (Ready to switch on)
xxxx xxxx x01x 0011	启动 (Switched on)
xxxx xxxx x01x 0111	操作使能 (Operation enabled)
xxxx xxxx x00x 0111	快速停机有效 (Quick stop active)
xxxx xxxx x0xx 1111	故障反应有效 (Fault reaction active)

程序中 ecat_loop 函数对上述状态机进行了切换。

```
{
  static int16_t i = 0;
  i++;
    if(flag_time == 1)
    {
      ec_send_processdata();
      ec_receive_processdata(EC_TIMEOUTRET);
      cur_status = inputs1->StatusWord;//0x6041
      switch(startup_step)
      {
        case 1:
          outputs1->ControlWord = 0x06;//0x6040
          if((cur_status==0x0631)||(cur_status==0x0231))
             startup_step=2;
          printf("0x06\n");
        break;
        case 2:
          outputs1->ControlWord = 0x07;
          if(cur_status==0x633)
             startup_step=3;
          printf("0x07\n");
          break;
        case 3:
          outputs1->ControlWord = 0x0f;
          if((cur_status==0x1637)||(cur_status==0x1633))
             startup_step=4;
          break;
        case 4:
           outputs1->ControlWord = 0x1f;
           outputs1->TargetPos = cur_pos;//0x607A

            csp_pos_delay++;
          if(csp_pos_delay <= 35)
            {
```

伺服反馈的状态信息并不尽相同，很多小伙伴调试的时候会卡在状态机无法切换，一直打印 0x06 或者 0x07 的情况。此种情况我们只需要观察返回的状态，若在如故障的情况下，只需在对应的状态机切换里面把反馈的值做一个或逻辑，则可顺利进入下一个状态机。

然后就可以通过 0x607a 对象字典发送位置信息了。

outputs1->TargetPos = cur_pos;//0x607A

如果电机转起来了，恭喜你！完成了主站电机控制的基本操作!

如果没转起来，请对照文档和程序仔细查看一下。参考说明书和 xml 文件。

5.3 混合（电机+IO）从站

如果有多个电机类从站，或者从站设备中既包含电机类从站，也包含 IO 类

从站。那么可以重复 5.2 中的操作，对每一个从站设备分别进行配置。IO 类从站的配置方式与电机类从站类似，5.2.1-5.2.4 的配置过程都一致。5.2.5 则不需要，配置完 5.2.1-5.2.4 后则可以对输入输出 IO 数据进行操作。

5.4 EtherCAT+TCP/IP 例程

提供的例程是冗余口支持 EtherCAT，主网口支持 LWIP TCP Server。例程用于控制普通 IO 模块，若需使用该方案控制伺服，ethercat 控制部分参考 5.2 的说明进行程序修改。

TCP 例程通过在 PC 上面打开网络调试工具，例程中网络参数为:

IP：192.168.1.12

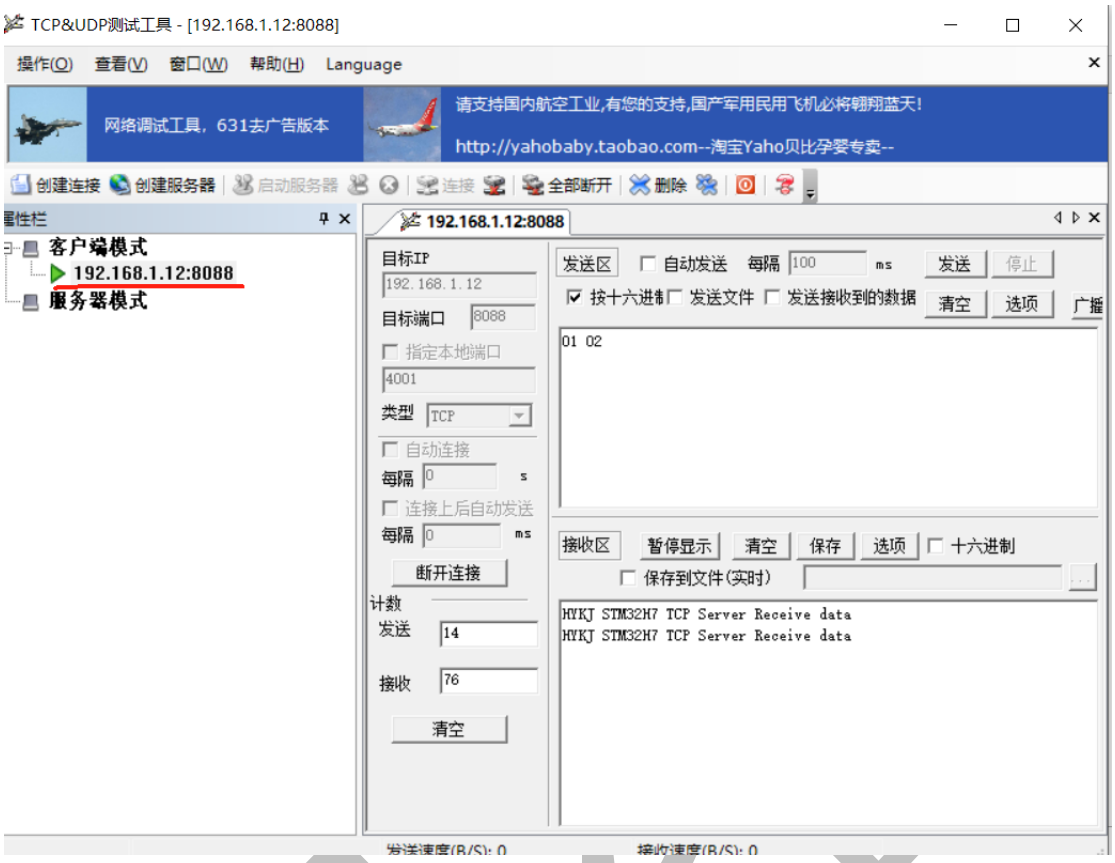
MASK:255.255.255.0

GATEWAY:192.168.1.1

PORT:8088

将电脑的 IP 参数设置为与开发板同一网段，设置 TCP&UDP 测试工具参数，如下图:

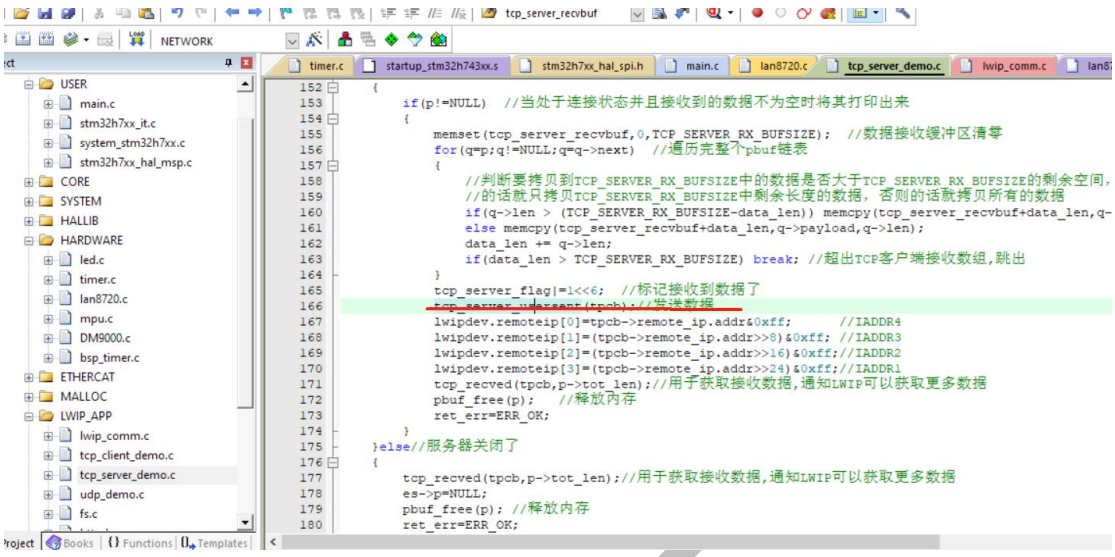
保密



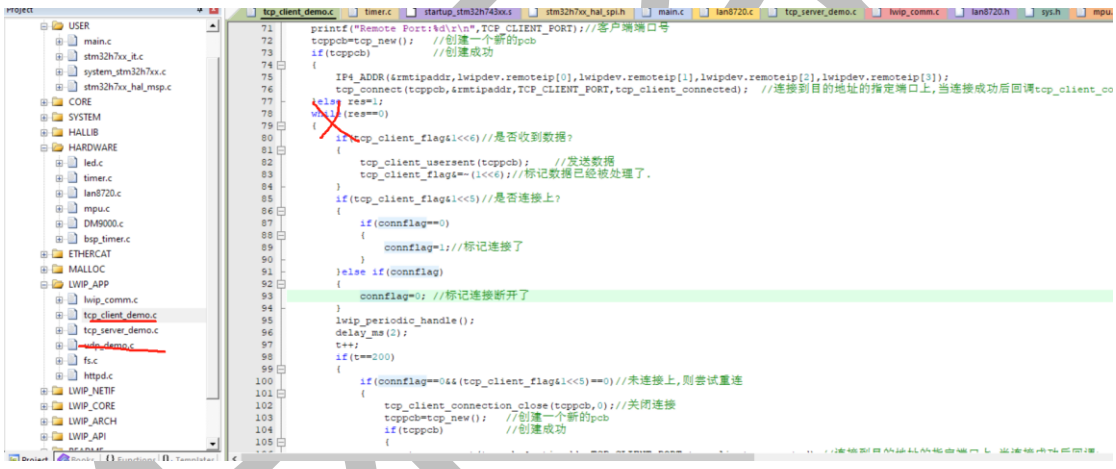
如下图中，可以看到程序中对收到的以太网数据进行了处理，然后通过 ethercat 协议控制从站开发板。做了一个简单的逻辑，如果收到的第一个字节非零，则停止 ethercat 跑马灯的运行。从上图中可以看到，每次发数据到开发板，则开发板会回一串字符，以表示接收到数据。

```
//定时器中断服务函数调用
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM1)
    {
        if(all_slave_op==1)  /* 所有从站已经处于OP状态 可以进行PDO的操作 */
        {
            pdoTimeFlag = 1;   /* PDO通讯定时标志位 */
            //跑马灯逻辑
            if(tcp_server_recvbuf[0]==0)
            {
                times_cnt++;
                if(times_cnt>10)
                {
                times_cnt=0;
                if(time_cnt<128)
                    time_cnt=time_cnt*2;
                else
                    time_cnt=1;
                }
            }
           ecat_loop(); /* 跑马灯产生的逻辑赋值到RxPDO */
         }
        else
        {
```

每次接收到数据，返回的数据程序中如下位置



该例程中只提供了 TCP server 的操作，如果有需要 可以使用 client 和 udp 的例程，只需在主循环中把相应屏蔽掉的地方解除。另外把其中的 while 循环参考 server 例程屏蔽掉即可。



6.其他

至此，开发板的介绍基本完成，该软件是在开源主站的程序上进行了裁剪。把其移植到 STM32 单片机上面。使 Ethercat 也能运行在嵌入式单片机上面。该程序仅起抛砖引玉的作用，供参考学习使用。若需了解更多关于主站的函数等，请查阅 SOEM 开源主站相关的代码。

By 合妍科技

--此后无正文

保密